LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 10
“FIND MAX/MIN”

DISUSUN OLEH:

AMMAR ATHAZZAM AL-AYYUBI

109082500012

S1 IF-13-02

DOSEN:

WAHYU ANDI SAPUTRA, S.PD., M.ENG.

PROGRAM STUDI S1 TEKNIK INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2025/2026

DASAR TEORI

1. Konsep Dasar Pencarian

- Pencarian adalah suatu proses yang sangat lazim dilakukan di dalam kehidupan sehari-hari, seperti pencarian file di dalam direktori komputer, teks di dalam dokumen, atau buku pada rak.
- Salah satu bentuk pencarian adalah algoritma untuk menemukan nilai terkecil atau terbesar pada sekumpulan data, yang biasa disebut sebagai pencarian nilai ekstrim.

2. Ide Algoritma Pencarian Nilai Ekstrim

- Ide dasar algoritmanya sangat sederhana dan memproses data secara sekuensial (berurutan).
- Nilai (atau indeks dari nilai tersebut) dari data yang telah diproses disimpan untuk kemudian dibandingkan dengan data-data berikutnya di dalam kumpulan data.
- Nilai yang pada akhirnya berhasil disimpan sampai algoritma tersebut berakhir adalah nilai ekstrim (maksimum/minimum) yang dicari.

3. Tahapan algoritma Secara Umum

Langkah-langkah pencarian nilai ekstrim adalah sebagai berikut:

1. Jadikan data pertama sebagai nilai ekstrim sementara.
2. Lakukan validasi nilai ekstrim dari data kedua hingga data terakhir.
3. Apabila nilai ekstrim sementara tersebut tidak valid (misal: ditemukan nilai yang lebih kecil saat mencari nilai minimum), maka *update* nilai ekstrim tersebut dengan data yang valid.
4. Apabila semua data telah selesai dicek, maka nilai ekstrim yang disimpan saat itu adalah nilai ekstrim yang valid.

4. Tipe Data dalam Pencarian

Pencarian nilai ekstrim dapat diimplementasikan pada dua jenis struktur array:

- **Tipe Data Dasar:** Misalnya mencari angka terkecil pada *array of integer*. Dalam bahasa Go, perlu diingat bahwa indeks array dimulai dari nol atau "0".
- **Tipe Data Terstruktur (*Struct*):** Digunakan untuk kasus yang lebih kompleks, seperti mencari data mahasiswa dengan nilai IPK terbesar, lagu dengan durasi terlama, atau pembalap dengan waktu tercepat.

5. Pentingnya Menyimpan "Indeks" Dibandingkan "Nilai"

- Pada proses pencarian, hal yang terpenting adalah mengetahui posisi atau indeks dari nilai yang dicari di dalam array kumpulan data.
- Terutama pada array bertipe terstruktur (*struct*), jika kita hanya mencari nilai terbesarnya saja, kita tidak akan memperoleh identitas data yang memiliki nilai tersebut.

- Oleh karena itu, sangat disarankan agar pencarian mengembalikan indeks dari nilai ekstrim tersebut. Dengan mendapatkan indeksnya, seluruh identitas data dapat diakses dan diketahui secara lengkap (misalnya `T[idx].nama`, `T[idx].nim`, hingga `T[idx].jurusan`).

SOAL LATIHAN

1. Latihan1

Source Code:

```
package main
import "fmt"
type arrKelinci [1000]float64

func main() {
    var n int
    var berat arrKelinci

    fmt.Scan(&n)
    fmt.Println("Jumlah kelinci:", n)

    for i := 0; i < n; i++ {
        fmt.Scan(&berat[i])
        fmt.Print("Berat kelinci ke-", i+1, ": ")
        fmt.Println(berat[i])
    }

    if n > 0 {
        var min float64 = berat[0]
        var max float64 = berat[0]

        for i := 1; i < n; i++ {
            if berat[i] < min {
                min = berat[i]
            }
            if berat[i] > max {
                max = berat[i]
            }
        }
        fmt.Printf("%.2f %.2f\n", min, max)
    }
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\golang\tugasweek10> go run 1.go
4 2 1 2 3
Jumlah kelinci: 4
Berat kelinci ke-1: 2
Berat kelinci ke-2: 1
Berat kelinci ke-3: 2
Berat kelinci ke-4: 3
1.00 3.00
PS D:\golang\tugasweek10>
```

Deskripsi Program:
Program ini digunakan untuk mendata berat anak kecil yang akan dijual ke pasar. Pertama program akan membaca input user jumah kelinci (n), dan berat masing masing kelinci ([i]), lemudian program akan memproses bilangan ril tersebut untuk mencari lalu program akan menampilkan dua bilangan rill yang menyatakan berat kelinci terkecil (min), dan terbesar (max).

2. Latihan2

Source Code:

```
package main
import "fmt"
type arrIkan [1000]float64

func main() {
    var x, y int
    var berat arrIkan

    fmt.Scan(&x, &y)

    for i := 0; i < x; i++ {
        fmt.Scan(&berat[i])
    }
    var totalWadah [1000]float64
    var countWadah int = 0
    var currentTotal float64 = 0
    var countIkan int = 0
    for i := 0; i < x; i++ {
        currentTotal += berat[i]
        countIkan++
        if countIkan == y || i == x-1 {
            totalWadah[countWadah] = currentTotal
            countWadah++
            currentTotal = 0
            countIkan = 0
        }
    }
    var sumTotal float64 = 0
    for i := 0; i < countWadah; i++ {
        fmt.Printf("%.2f ", totalWadah[i])
        sumTotal += totalWadah[i]
    }
    fmt.Println()
    if countWadah > 0 {
        rataRata := sumTotal / float64(countWadah)
        fmt.Printf("%.2f\n", rataRata)
    }
}
```

Output:

```
PS D:\golang\tugasweek10> go run 2.go
5 2
1.5 2.0 1.0 3.0 2.5
3.50 4.00 2.50
3.33
PS D:\golang\tugasweek10>
```

Deskripsi Program:
Program ini digunakan untuk menentukan tarif ikan yang akan di jual ke pasar. Program array berkapasitas 1000 untuk menampung data berat ikan. Setelah menrima masukan berupa banyaknya ikan dan kapasitas perwadah, program akan menghitun total berat ikan di setiap wadah dan menampilkan berat rata rata ikan di setiap wadah tersebut.

3. Latihan3

Source Code:

```
package main
import "fmt"

type arrBalita [100]float64
func hitungMinMax(arrBerat arrBalita, n int, bMin, bMax
*float64) {
    *bMin = arrBerat[0]
    *bMax = arrBerat[0]

    for i := 1; i < n; i++ {
        if arrBerat[i] < *bMin {
            *bMin = arrBerat[i]
        }
        if arrBerat[i] > *bMax {
            *bMax = arrBerat[i]
        }
    }
}
func rerata(arrBerat arrBalita, n int) float64 {
    var total float64 = 0
    for i := 0; i < n; i++ {
        total += arrBerat[i]
    }
```

```go
    return total / float64(n)
}

func main() {
    var n int
    var berat arrBalita
    var bMin, bMax float64

    fmt.Print("Masukan banyak data berat balita: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Printf("Masukan berat balita ke-%d: ", i+1)
        fmt.Scan(&berat[i])
    }

    if n > 0 {
        hitungMinMax(berat, n, &bMin, &bMax)
        rataRata := rerata(berat, n)

        fmt.Printf("Berat balita minimum: %.2f kg\n",
bMin)
        fmt.Printf("Berat balita maksimum: %.2f kg\n",
bMax)
        fmt.Printf("Rerata berat balita: %.2f kg\n",
rataRata)
    }
}
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS D:\golang\tugasweek10> go run 3.go
Masukan banyak data berat balita: 4
Masukan berat balita ke-1: 5.3
Masukan berat balita ke-2: 6.2
Masukan berat balita ke-3: 4.1
Masukan berat balita ke-4: 9.9
Berat balita minimum: 4.10 kg
Berat balita maksimum: 9.90 kg
Rerata berat balita: 6.38 kg
PS D:\golang\tugasweek10>
```

Deskripsi Program:
Program ini digunakan untuk mencatat data berat balita di posyandu.
Pertama user akan mendata jumlah bayi di posyandu, lalu program akan memanggil subprogram untuk menghitung dan menampilkan berat balita terkecil, berat balita terbesar, dan ratarata berat balita.